

Comparaison des Méthodes de Détection et de Description de Points d'Intérêt pour l'Alignement d'Images en Environnement Ferroviaire

Julian Gosselin

Octobre 2025

Table des matières

1	Introduction	3
2	Fondements de la détection et de la description de points d'intérêt	3
3	Méthodes classiques	3
3.1	Harris	4
3.2	FAST (Features from Accelerated Segment Test)	5
3.3	SIFT (Scale-Invariant Feature Transform)	5
3.4	SURF (Speeded-Up Robust Features)	6
3.5	KAZE / AKAZE	7
3.6	Remarques pratiques sur les méthodes classiques	7
4	Méthodes binaires légères	8
4.1	BRIEF (Binary Robust Independent Elementary Features)	8
4.2	BRISK (Binary Robust Invariant Scalable Keypoints)	9
4.3	ORB (Oriented FAST and Rotated BRIEF)	10
4.4	FREAK (Fast Retina Keypoint)	11
4.5	Notes d'implémentation communes et bonnes pratiques	12
4.6	Résumé pratique pour l'application ferroviaire	12
5	Méthodes apprises par deep learning	13
5.1	TILDE (Temporally Invariant Learned DETector)	13
5.2	LIFT (Learned Invariant Feature Transform)	14

5.3	SuperPoint	14
5.4	D2-Net	16
5.5	R2D2 (Repeatable and Reliable Detector and Descriptor)	16
5.6	DISK	16
5.7	ALIKE	17
5.8	Comparaison synthétique et recommandations pratiques	17
6	Comparaison globale des méthodes	18
7	Analyse pour le projet ferroviaire	19
8	Conclusion	20

1 Introduction

L'enregistrement d'images (*image registration*) est un processus fondamental en vision par ordinateur. Il consiste à aligner plusieurs images d'une même scène prises à différents instants, points de vue ou sous des conditions lumineuses distinctes. Dans le cadre d'un système de surveillance ferroviaire, ce problème se traduit par la nécessité d'aligner des images capturées quotidiennement par une caméra embarquée sur un train, afin de comparer l'état des infrastructures (rails, ballast, signalisations, etc.) au fil du temps.

L'un des défis majeurs est de réaliser cet alignement en temps réel ou quasi temps réel, sur un système embarqué, malgré des variations importantes d'éclairage, de perspective ou de conditions météo. Ce rapport présente une étude approfondie des principales méthodes de détection et de description de points d'intérêt, depuis les approches classiques (SIFT, SURF, ORB) jusqu'aux réseaux neuronaux modernes (SuperPoint, R2D2, ALIKE).

2 Fondements de la détection et de la description de points d'intérêt

Un pipeline classique d'enregistrement d'images comprend quatre étapes :

1. Détection de points d'intérêt (keypoints) stables dans les images.
2. Description locale de chaque point par un vecteur caractéristique invariant (descripteur).
3. Appariement (matching) entre les descripteurs des deux images.
4. Estimation de la transformation géométrique (ex. homographie) via un algorithme robuste (RANSAC).

Les détecteurs doivent produire des points *répétables* et *distinctifs*, tandis que les descripteurs doivent être invariants aux changements d'échelle, de rotation et d'éclairage. L'équilibre entre robustesse et vitesse est central pour une application embarquée.

3 Méthodes classiques

Les méthodes dites « classiques » regroupent des détecteurs et des descripteurs construits à partir d'analyses locales d'intensité et de gradients, fondées sur des principes mathématiques explicites (dérivées, matrices de structure, espace d'échelle gaussien, filtrage linéaire ou non linéaire, etc.). Elles sont qualifiées de classiques car elles précèdent l'ère des approches entièrement apprises (deep learning) et restent encore aujourd'hui des références en raison de leur interprétabilité, de leur implémentation simple et de leur robustesse analytique. Ces méthodes forment la base des pipelines de vision « feature-based » : détection → description → appariement → estimation géométrique.

Les propriétés typiques qui caractérisent ces méthodes sont :

- une formulation analytique (gradients, hessiennes, réponses filtrées) permettant d’expliquer pourquoi un point est détecté ;
- des invariances obtenues par construction (ex. invariance à la rotation via histogrammes d’orientations, invariance à l’échelle via un espace multi-échelle) ;
- un coût de calcul souvent non négligeable (ex. SIFT), mais predictable et optimisable (ex. SURF, AKAZE) ;
- une forte disponibilité logicielle (OpenCV, VLFeat, etc.), facilitant l’intégration.

Dans les paragraphes qui suivent, nous détaillons le principe de fonctionnement, les formules clefs, et les avantages / limites de chaque méthode classique retenue.

3.1 Harris

Le détecteur de Harris (Harris & Stephens, 1988) identifie les coins en analysant la variation de l’erreur d’approximation d’un patch quand on lui applique un petit déplacement. Soit $I(x, y)$ l’image et $w(x, y)$ un noyau de pondération (typiquement une gaussienne). On définit la matrice de second-moment (ou *structure tensor*) :

$$M(x, y) = \begin{pmatrix} A & C & C & B \end{pmatrix} \text{ avec } A = w * I_x^2, B = w * I_y^2, C = w * (I_x I_y),$$

où I_x, I_y sont les dérivées partielles spatiales et $*$ la convolution. L’idée est d’étudier les valeurs propres λ_1, λ_2 de M : un coin correspond à λ_1 et λ_2 grandes (fortes variations dans deux directions). La réponse de Harris classique est donnée par :

$$R = \det(M) - k (\text{trace}(M))^2 = AB - C^2 - k(A + B)^2,$$

avec un paramètre k typiquement 0.04–0.06. On détecte comme coins les points où R est un maximum local et dépasse un seuil.

Points techniques et améliorations :

- calculs des dérivées via filtres de Sobel ou dérivées gaussiennes pour robustesse au bruit ;
- suppression non maximale (NMS) sur la carte R pour localiser les maxima ;
- sensibilité au bruit : il est courant d’appliquer un lissage préalable (gaussien) et d’ajuster la taille de la fenêtre w pour compenser.

Avantages / Inconvénients : Harris est rotation-invariant (la matrice M encode la forme locale) et peu coûteux pour des images de taille modérée ; il ne fournit pas d’invariance à l’échelle (pas de scale-space) et est sensible au bruit et au flou.

3.2 FAST (Features from Accelerated Segment Test)

FAST (Rosten & Drummond) est un détecteur conçu pour la rapidité maximale. Pour un pixel candidat p on considère un cercle de 16 pixels autour de p (rayon généralement 3). p est un coin si au moins n pixels consécutifs sur ce cercle sont tous plus clairs que $I(p) + t$ ou tous plus sombres que $I(p) - t$ (classiquement $n = 12$ pour FAST-12). Pour accélérer la décision, on teste d'abord des positions stratégiques (pixels 1,5,9,13) : si elles ne vérifient pas la condition, on rejette rapidement p .

Optimisations :

- utilisation d'un arbre de décision appris (decision tree) pour ordonner les tests et atteindre une grande efficacité (FAST-ER, FASTdetector) ;
- non-maxima suppression sur la réponse binaire ; pour ordonner les points on peut définir un score (ex. somme des différences absolues sur le cercle) ou combiner FAST avec la mesure de coin de Harris pour prioriser les points les plus stables.

Avantages / Inconvénients : extrême rapidité (idéal pour pipelines temps réel), implémentation simple ; cependant FAST n'ignore pas l'échelle (il détecte à une seule échelle) et peut produire des faux positifs le long d'arêtes ou sous bruit. Il est souvent combiné avec une autre méthode pour augmenter la fiabilité.

3.3 SIFT (Scale-Invariant Feature Transform)

SIFT vise l'invariance robuste à l'échelle et à la rotation via un espace d'échelle gaussien. Les étapes clefs :

1. Construction du scale-space On calcule $L(x, y, \sigma) = G(x, y, \sigma) * I(x, y)$, où G est une gaussienne de variance σ^2 . Les différences de gaussiennes (DoG) sont :

$$D(x, y, \sigma) = L(x, y, k\sigma) - L(x, y, \sigma),$$

les maxima / minima de D dans l'espace (x, y, σ) constituent des candidats keypoints.

2. Localisation précise des keypoints On ajuste la position par une approximation quadratique de D (Taylor expansion) autour du candidat pour obtenir un décalage $\hat{\mathbf{x}}$:

$$\hat{\mathbf{x}} = - \left(\frac{\partial^2 D}{\partial \mathbf{x}^2} \right)^{-1} \frac{\partial D}{\partial \mathbf{x}}.$$

On élimine les points à faible contraste (seuil sur $D(\hat{\mathbf{x}})$) et les réponses d'arêtes via la relation entre trace et déterminant de la Hessienne locale H : on rejette si

$$\frac{\text{trace}(H)^2}{\det(H)} > \frac{(r+1)^2}{r}$$

avec r un ratio empirique (par ex. $r = 10$).

3. Orientation On calcule pour chaque point les gradients dans un voisinage (magnitudes $m(x, y)$ et orientations $\theta(x, y)$) :

$$m = \sqrt{I_x^2 + I_y^2}, \quad \theta = \text{atan2}(I_y, I_x).$$

On construit un histogramme d'orientation (36 bins couvrant 360°) pondéré par la magnitude et un noyau gaussien ; la(les) orientation(s) dominante(s) est/sont assignée(s) au keypoint.

4. Descripteur Autour du point orienté on extrait une fenêtre (par ex. 16×16) divisée en 4×4 sous-régions ; pour chaque sous-région on calcule un histogramme d'orientations à 8 bins, ce qui donne $4 \times 4 \times 8 = 128$ valeurs normalisées $\rightarrow$ vecteur final (128D). Normalisations et seuillage des valeurs extrêmes améliorent l'invariance photométrique.

Avantages / Inconvénients : SIFT est extrêmement robuste (échelle, rotation, illumination modérée) et standard dans de nombreux benchmarks ; son coût (filtrage multi-échelle, localisation fine, calcul du descripteur) est élevé, ce qui le rend inadapté pour des systèmes embarqués temps réel.

3.4 SURF (Speeded-Up Robust Features)

SURF reprend les idées de SIFT mais remplace des opérations coûteuses par des approximations rapides.

Principes :

- utilisation d'*integral images* pour calculer rapidement des convolutions par des box-filters quelle que soit la taille du filtre (coût constant par filtre) ;
- approximation du détecteur Hessien via des filtres en boîtes tridimensionnels (D_x , D_y , D_{xy} approximés) ; on détecte les extrema de la réponse déterminant approximée $\det(\tilde{H})$;
- estimation de l'orientation par l'intégration des réponses Haar-wavelet dans un voisinage circulaire (réponse sur x et y dans un secteur) ;
- descripteur construit sur une région orientée et subdivisée en 4×4 sous-régions, pour chaque sous-région on somme les réponses Haar $\sum dx, \sum |dx|, \sum dy, \sum |dy|$, donnant typiquement un vecteur 64D (SURF-64).

Avantages / Inconvénients : SURF offre une très bonne compromission précision/vitesse (plus rapide que SIFT) et reste robuste aux variations d'échelle et rotation ; toutefois le recours à des approximations implique parfois une légère perte de précision par rapport à SIFT. SURF a été largement optimisé dans les bibliothèques (OpenCV).

3.5 KAZE / AKAZE

KAZE (Alcantarilla et al.) propose de construire l'espace d'échelle non linéaire en utilisant une diffusion non linéaire (permettant de préserver les contours et détails) au lieu de la diffusion gaussienne linéaire. La PDE de base s'écrit :

$$\frac{\partial L}{\partial t} = \text{div}(c(x, y, t) \nabla L),$$

où $c(x, y, t)$ est la *fonction de conduction* dépendant du gradient $|\nabla L|$ (ex. fonctions de Perona–Malik) :

$$c(\cdot) = g(|\nabla L_\sigma|), \quad g_1(s) = \exp(-s^2/k^2), \quad g_2(s) = \frac{1}{1+s^2/k^2}, \text{ etc.}$$

Cette formulation produit un espace d'échelle où les structures de bord sont mieux conservées. KAZE implémente des schémas numériques pour résoudre efficacement cette PDE ; AKAZE (Accelerated KAZE) utilise un schéma Fast Explicit Diffusion (FED) et une approximation multi-échelle pour rendre la méthode pratique.

Descripteurs associés : AKAZE propose des descripteurs binaires basés sur le LDB (Local Difference Binary) et une variante M-LDB adaptée à l'espace non linéaire, fournissant des descripteurs compacts et invariants.

Avantages / Inconvénients : KAZE/AKAZE donnent une bonne sensibilité aux structures locales (meilleure préservation des contours) et robustesse au bruit/illumination ; leur coût est supérieur à SURF dans sa version pure, mais AKAZE rend l'approche utilisable en temps raisonnable. Ils sont particulièrement utiles quand la préservation des contours est critique (ex. infrastructures avec arêtes marquées).

3.6 Remarques pratiques sur les méthodes classiques

- **Multi-échelle** : seules SIFT, SURF, KAZE/AKAZE intègrent explicitement une notion d'échelle. Les détecteurs rapides (FAST, Harris) doivent être appliqués sur une pyramide d'images si l'on veut détection multi-échelle.
- **Orientation** : SIFT et SURF assignent explicitement une orientation au point, permettant descripteurs rotation-invariants ; ORB (voir section suivante) calcule une orientation simple via le centroïde d'intensité.
- **Implémentation et disponibilité** : toutes les méthodes évoquées disposent d'implémentations optimisées (OpenCV) et sont donc simples à tester/évaluer.
- **Coût empirique** : en pratique, pour une image 640×480 sur CPU moderne : FAST/ORB/BRISK opèrent en quelques millisecondes, SURF/KAZE/AKAZE prennent quelques dizaines de ms, et SIFT peut atteindre plusieurs dizaines à centaines de

ms selon implémentation et paramètres.

4 Méthodes binaires légères

Les méthodes binaires légères constituent un sous-ensemble de techniques *feature-based* conçues pour minimiser le coût de calcul et la taille mémoire des descripteurs. Elles partagent la même philosophie : remplacer des vecteurs flottants (histogrammes de gradients) par des vecteurs binaires obtenus par des comparaisons simples d'intensité. Ceci permet des comparaisons ultra-rapides via la *distance de Hamming* (opérations XOR + popcount). Ces méthodes sont largement employées dans des systèmes embarqués ou temps réel où la latence et la mémoire sont contraintes. Elles offrent un très bon compromis vitesse/mémoire au prix d'une invariance parfois limitée (rotation, échelle) que divers raffinements cherchent à corriger.

Dans les paragraphes suivants, nous détaillons BRIEF, BRISK, ORB et FREAK : principe, formules clefs, pratique d'implémentation, performances et pertinence pour l'alignement d'images ferroviaire.

4.1 BRIEF (Binary Robust Independent Elementary Features)

BRIEF est l'un des premiers descripteurs binaires simples et efficaces.

Principe Pour un point clé centré à la position p , on considère un patch carré autour de p (ex. 31×31). On définit un ensemble de N_d paires de positions de pixels (x_i, y_i) dans ce patch. Le bit b_i du descripteur est obtenu par la comparaison :

$$b_i = \tau(I; x_i, y_i) = \{ 1 \text{ si } I(x_i) < I(y_i), 0 \text{ sinon},$$

où $I(\cdot)$ donne l'intensité (après éventuellement un léger lissage). Le descripteur final est le vecteur binaire $[b_1, \dots, b_{N_d}]$ (typiquement $N_d = 128$ ou 256).

Caractéristiques pratiques

- très faible empreinte mémoire : 128 bits = 16 octets ; 256 bits = 32 octets ;
- comparaison très rapide : distance de Hamming calculable par XOR + popcount (instructions processeur optimisées) ;
- robustesse raisonnable au bruit local si le patch est préfiltré (gaussien) ;
- sensibilité élevée à la rotation et à l'échelle : le pattern de tests est fixe dans les coordonnées du patch.

Améliorations courantes Pour pallier la sensibilité à la rotation, BRIEF est souvent utilisé conjointement à une estimation d’orientation (ex. SIFT-like orientation) ou avec un détecteur orienté ; néanmoins ces solutions augmentent le coût. Une autre amélioration consiste à sélectionner statistiquement les paires (x_i, y_i) les plus discriminantes (variance proche de 0.5 et faible corrélation) — idée reprise par ORB.

Avantages / Inconvénients

- + : simplicité, très faible coût mémoire/CPU, appariement ultrarapide.
- - : pas d’invariance intrinsèque à la rotation/échelle ; moins robuste que des descripteurs float (SIFT) en présence de grandes transformations.

4.2 BRISK (Binary Robust Invariant Scalable Keypoints)

BRISK vise à combiner rapidité et invariance à la rotation et à l’échelle.

Principe BRISK utilise :

1. un détecteur basé sur FAST appliqué sur une pyramide multi-échelle (octaves et intra-octaves) pour trouver des points à différentes tailles ;
2. un schéma d’échantillonnage circulaire concentrique (figure de motifs) : N points d’échantillonnage disposés sur plusieurs cercles autour du keypoint ;
3. une séparation des paires en *long-distance* (sub-ensemble L) et *short-distance* (S). Les paires longues sont utilisées pour estimer la direction dominante (orientation) et les paires courtes pour construire les bits du descripteur.

La direction $\mathbf{g}$ du patch est obtenue en moyennant les contributions de paires longues :

$$\mathbf{g} = \frac{1}{|L|} \sum_{(p_i, p_j) \in L} g(p_i, p_j),$$

où une forme possible de $g(p_i, p_j)$ (basée sur les intensités lissées $I(\cdot, \sigma)$) est donnée par la littérature comme

$$g(p_i, p_j) = \frac{(p_i - p_j)(I(p_j, \sigma_j) - I(p_i, \sigma_i))}{\|p_i - p_j\|^2}.$$

L’angle d’orientation $\alpha = \text{atan2}(g_y, g_x)$ permet de faire tourner le motif d’échantillonnage avant de calculer les comparaisons pour obtenir une invariance à la rotation.

Descripteur Les bits sont construits en comparant les intensités de paires (p_i^α, p_j^α) du motif tourné (paires courtes principalement), exactement comme BRIEF mais dans les coordonnées orientées. BRISK produit typiquement un descripteur de 512 bits.

Caractéristiques pratiques

- invariance à la rotation obtenue explicitement ; l'invariance à l'échelle est assurée par la détection multi-échelle ;
- empreinte mémoire : 512 bits = 64 octets ;
- coût : plus élevé que BRIEF simple (calcul de la pyramide, estimation orientation), mais toujours adapté au temps réel sur CPU moyen.

Avantages / Inconvénients

- + : bonne robustesse rotation/échelle relative, descripteur dense et discriminant pour une méthode binaire.
- - : coût et complexité supérieurs à BRIEF/ORB ; taille mémoire plus importante (512 bits).

4.3 ORB (Oriented FAST and Rotated BRIEF)

ORB est une méthode pragmatique très populaire, combinant la rapidité de FAST pour la détection et la compacité de BRIEF pour la description, avec des correctifs pour l'orientation et la qualité des tests.

Pipeline ORB (principales étapes)

1. **Détection** : FAST (généralement FAST-9) est exécuté sur une pyramide d'images pour obtenir des points multi-échelle.
2. **Scoring** : les points FAST sont ordonnés par un score (souvent le score de Harris appliqué localement) et l'on ne garde que les N meilleurs par niveau pour contrôler le nombre de points.
3. **Orientation (oFAST)** : pour chaque patch centré sur le keypoint on calcule les moments d'intensité :

$$m_{pq} = \sum_{x,y} x^p y^q I(x,y),$$

puis le centroïde $C = (m_{10}/m_{00}, m_{01}/m_{00})$. L'angle d'orientation est

$$\theta = \text{atan2}(m_{01}, m_{10}).$$

4. **Description (rBRIEF)** : le motif de tests BRIEF est tourné de l'angle θ ; on évalue alors les N_d comparaisons aux positions tournées pour former le vecteur binaire.
5. **Sélection apprise des tests** : ORB ne prend pas des paires aléatoires comme BRIEF mais sélectionne (hors-entraînement) 256 paires discriminantes via un processus d'apprentissage offline : on génère toutes les paires candidates, on trie par

variance (moyenne proche de 0.5), puis on applique une sélection gloutonne en éliminant les tests fortement corrélés pour obtenir un ensemble de tests dé-corrélés et informatifs.

Comparaison et matching

- la comparaison entre deux descripteurs ORB se fait par la distance de Hamming ;
- appariement : on utilise souvent un *brute force* Hamming + ratio-test (distance 1ère / distance 2ème) ou un matching mutuel (cross-check) ; l'opération est extrêmement efficace (XOR + popcount).

Caractéristiques pratiques

- taille : 256 bits (32 octets) ; bon compromis entre discriminativité et mémoire ;
- vitesse : très rapide sur CPU (détection FAST + calcul rBRIEF en quelques millisecondes par image selon résolution et paramètres) ; appariement Hamming très rapide ;
- robustesse : rotation-invariant (par oFAST + rBRIEF), tolère mieux l'éclairage ; invariance à l'échelle limitée sauf si on utilise suffisamment de niveaux de pyramide.

Avantages / Inconvénients

- + : excellent pour systèmes embarqués sans accélération GPU, large adoption (OpenCV), paramétrage simple, matching ultra-rapide.
- - : moins robuste que SIFT/AKAZE ou des méthodes deep-learned sur transformations sévères (day/night, fortes vues obliques) ; l'invariance à l'échelle reste approximative.

4.4 FREAK (Fast Retina Keypoint)

FREAK s'inspire de la structure rétinienne et propose une grille d'échantillonnage non uniforme (densité plus élevée au centre) et des récepteurs de différentes tailles (coarse-to-fine).

Principe

- motif d'échantillonnage en anneaux imitant la fovéa/para-fovéa/peri-fovéa ; chaque position correspond à une région filtrée (récepteur) ;
- sélection des paires de récepteurs par critère d'information (variance élevée, faible corrélation) pour composer un vecteur binaire (souvent 512 bits) ;
- estimation d'orientation via des différences pondérées entre récepteurs symétriques.

Caractéristiques pratiques

- robustesse aux changements d'échelle et à la rotation grâce au motif et à l'estimation d'orientation ;
- empreinte : typiquement 512 bits (64 octets) ; appariement Hamming ;
- très rapide en extraction et en matching ; bon comportement sur scènes naturelles et structurelles.

Avantages / Inconvénients

- + : compact, rapide, bonne robustesse pratique ; souvent performant en complément d'un détecteur FAST/ORB.
- - : légèrement moins discriminant que descripteurs appris modernes ; performances dégradées sur textures très fines ou changements extrêmes d'apparence.

4.5 Notes d'implémentation communes et bonnes pratiques

- **Prétraitement** : un léger lissage (gaussien) avant le calcul des comparaisons améliore la robustesse au bruit ; on normalise parfois l'intensité locale pour réduire les effets d'exposition.
- **Pyramide d'images** : pour obtenir une invariance à l'échelle, appliquer le détecteur sur plusieurs niveaux d'une pyramide d'images et propager la détection au descripteur (ORB, BRISK).
- **Sélection des points** : limiter le nombre de keypoints (ex. 500–2000) par image pour tenir le budget temps ; trier par score (Harris ou réponse BRISK) et répartir spatialement (grid) pour éviter la concentration sur une zone.
- **Matching** : utiliser le *Hamming distance* + cross-check ou ratio-test ; implémentations matérielles fournissent souvent des accélérations pour le popcount.
- **RANSAC et filtrage géométrique** : toujours inclure une étape de RANSAC sur les matches bruts pour estimer l'homographie ou la transformation rigide et rejeter les outliers.

4.6 Résumé pratique pour l'application ferroviaire

- **ORB** : premier choix si contrainte CPU-only. Rapide, rotation-invariant, faible empreinte mémoire ; espérance de tenir un pipeline quasi temps réel sur processeurs embarqués (ARM) si on limite le nombre de points.
- **BRISK / FREAK** : bonnes alternatives si l'on veut améliorer invariance à l'échelle (BRISK) ou robustesse pratique (FREAK). Légèrement plus coûteux en mémoire que ORB.
- **BRIEF seul** : utile uniquement si on a déjà une méthode d'orientation fiable ; sinon trop sensible aux rotations.

- **Conseil opérationnel** : pour l’alignement journalier des images de rails (trajectoire répétée, variations lumineuses), une configuration ORB (detect : FAST pyramidé + score Harris, describe : rBRIEF) avec un appariement Hamming + RANSAC est un bon point de départ. Si tu peux embarquer un GPU/NPU, il est pertinent de considérer ensuite descripteurs appris (SuperPoint / ALIKE) pour améliorer la robustesse sous conditions extrêmes (jour/nuit, saisons).

5 Méthodes apprises par deep learning

Elles tendent à offrir une meilleure répétabilité et une plus grande robustesse aux variations photométriques et géométriques que les descripteurs classiques, au prix d’un coût de calcul et d’un effort d’entraînement plus élevés. Les paragraphes qui suivent détaillent les architectures et les principes d’entraînement des méthodes les plus répandues ou pertinentes pour notre problématique.

5.1 TILDE (Temporally Invariant Learned DEtector)

Principe TILDE est un détecteur supervisé conçu pour maximiser la *répétabilité temporelle* des keypoints sous variations d’éclairage. Le jeu d’entraînement est construit à partir de séquences d’images acquises par des webcams (variations jour/nuit, météorologie) : les positions récurrentes détectées (par exemple via SIFT puis moyennées) servent de pseudo-labels. Le modèle est un régresor (piecewise-linear regressor ou petite CNN) qui prédit une carte de scores $S(x,y)$.

Fonction de perte La perte combine trois termes :

$$\mathcal{L} = \mathcal{L}_c + \lambda_s \mathcal{L}_s + \lambda_t \mathcal{L}_t,$$

où $\mathcal{L}_c$ est une perte de type classification (séparer positives/négatives), $\mathcal{L}_s$ régularise la forme/compacité de la réponse (shape regularization) et $\mathcal{L}_t$ force la stabilité temporelle (temporal regularization) en favorisant la répétabilité des maxima sur les images temporellement alignées.

Propriétés et usage

- Excellente robustesse aux variations d’éclairage ; conçue pour détecter des points souvent visibles sur le même emplacement malgré la lumière variable.
- Limites sur invariance géométrique (rotation/échelle) : TILDE est moins performant quand les vues diffèrent fortement en perspective.
- Déploiement : léger (régresor peu profond) → utilisable sur CPU embarqué si nécessaire.

5.2 LIFT (Learned Invariant Feature Transform)

Principe LIFT est un pipeline appris de bout en bout composé de trois modules : *Detector*, *Orientation Estimator* et *Descriptor*. L'idée est d'apprendre successivement (et finalement combiner) des composants qui reproduisent les étapes SIFT mais optimisées par apprentissage.

Entraînement et soft-argmax

- Le descripteur est entraîné d'abord (Siamese / triplet losses) sur paires/triplets de patches correspondants/non correspondants.
- L'estimateur d'orientation est entraîné pour minimiser la distance entre descripteurs de patches orientés différemment.
- Le détecteur est entraîné en dernier, utilisant un *soft-argmax* pour extraire une localisation subpixelle à partir d'une carte de score S :

$$\mathbf{x} = \frac{\sum_{\mathbf{y}} \exp(\beta S(\mathbf{y})) \mathbf{y}}{\sum_{\mathbf{y}} \exp(\beta S(\mathbf{y}))},$$

où β contrôle la "sharpness" du soft-argmax.

Perte Les pertes incluent des pertes contrastives/triplet pour le descripteur et des pertes de régressions pour l'orientation et la localisation. L'approche vise la robustesse aux vues et à l'échelle.

Propriétés et usage

- Très robuste et conceptuellement « SIFT appris » (détection + orientation + description).
- Inconvénient majeur : traitement patch-wise et pipeline complexe $\rightarrow$ inference lente comparée aux méthodes fully-convolutional ; difficile à mettre en œuvre en réel sans accélération.
- Intérêt : bonne option pour off-line training/benchmarks, moins adapté pour embarqué strict sans NPU/GPU.

5.3 SuperPoint

Architecture et principe SuperPoint est un réseau fully-convolutional « two-headed » qui calcule en une seule passe une carte de détection et une carte de descripteurs semi-denses. L'encodeur est léger (blocs convolutifs + pooling, réduction spatiale par 8) ; les décodeurs sont non-apprenants en upsampling pour rester rapides. Détails pratiques :

- **Tête détecteur** : sortie $H/8 \times W/8 \times 65$; après softmax spatial et reshaping on obtient une carte de probabilité de keypoint à la résolution d'origine (canal

supplémentaire "fond").

- **Tête descripteur** : sortie $H/8 \times W/8 \times D$ (typiquement $D = 256$), puis up-sampling non-appris + interpolation bilinéaire pour obtenir descripteurs à position arbitraire, suivie d'une normalisation L2.

Entraînement : MagicPoint & Homographic Adaptation

1. **MagicPoint** : pré-entraînement sur dataset synthétique (formes géométriques avec coins/clés parfaits) pour apprendre à repérer des coins bien définis.
2. **Homographic Adaptation** : génération de pseudo-labels sur images réelles : on applique un grand nombre d'homographies aléatoires à une image, on détecte points via MagicPoint sur chaque version et on agrège (vote / average) pour obtenir des labels robustes.
3. **Joint training** : réseau final est entraîné end-to-end avec une perte combinée :

$$\mathcal{L} = \mathcal{L}_p + \lambda \mathcal{L}_d,$$

où $\mathcal{L}_p$ est la perte de détection (cross-entropy / softmax sur les 65 canaux) et $\mathcal{L}_d$ une perte discriminative pour les descripteurs (hinge/contrastive/triplet selon implémentation).

Performances et caractéristiques pratiques

- **Vitesse** : inférence très rapide sur GPU (mesures publiées ~ 11.2 ms pour l'inférence sur Titan X, interpolation des descripteurs ~ 1.5 ms $\rightarrow$ total ~ 13 ms $\rightarrow \approx 70$ FPS en 480×640). Sur CPU l'inférence est beaucoup plus lente (dépend fortement du backbone et de l'optimisation).
- **Descripteur** : 256-D float L2-normalisé (semi-dense $\rightarrow$ échantillonné aux coordonnées de keypoints).
- **Robustesse** : excellente répétabilité, bonne tolérance aux variations d'éclairage grâce à l'adaptation homographique ; invariance aux rotations modérées via l'entraînement / augmentations, mais absence d'un mécanisme explicite d'orientation comme SIFT.
- **Déploiement** : recommandé sur GPU/NPU embarqué (ex. Jetson), quantification FP16/INT8 et pruning permettent d'améliorer le throughput pour des plateformes plus modestes.

Conseil pratique Pour un pipeline temps réel embarqué, SuperPoint est un excellent choix si une accélération matérielle est disponible. Le réglage le plus impactant est la résolution d'entrée (réduite pour gagner en vitesse) et la taille des descripteurs si l'on souhaite alléger la mémoire.

5.4 D2-Net

Principe D2-Net exploite les activations d'un backbone CNN (généralement VGG) pour produire *dense feature maps*. Les points d'intérêt sont extraits comme maxima locaux dans ces cartes d'activation et les descripteurs sont simplement les vecteurs d'activation associés. L'approche est *dense-by-design* : elle traite globalement l'image plutôt que patch-wise ou grille coarse-to-fine.

Propriétés

- Très robuste sur conditions difficiles (jour/nuit, forts changements de point de vue) car les représentations apprises sont riches et multi-échelle implicitement.
- Descripteurs souvent plus larges (ex. 512-D) et coûteux en mémoire.
- **Inconvénient majeur** : coût d'inférence élevé, extraction dense → souvent 1 FPS ou moins sur GPU si non optimisé ; difficilement utilisable en temps réel embarqué.

5.5 R2D2 (Repeatable and Reliable Detector and Descriptor)

Principe R2D2 prédit trois sorties : une carte de descripteurs, une carte de *repeatability* (indiquant si un emplacement est répétable) et une carte de *reliability* (indiquant si le descripteur associé est discriminant). L'entraînement optimise à la fois la répétabilité et la fiabilité, en forçant le réseau à sélectionner des points qui sont à la fois détectables et utiles pour le matching.

Perte et objectifs Les pertes combinent :

- une perte de répétabilité (encourage maxima stables entre images géométriquement transformées) ;
- une perte de fiabilité (favorise descripteurs différentiables entre points différents) ;
- une perte de distance (contrastive/triplet) pour les descripteurs.

Caractéristiques pratiques

- Descripteur compact (ex. 128-D) ; bon équilibre précision / coût mémoire.
- Très bonnes performances sur datasets de localisation extérieur (Aachen, Mega-Depth) — robustesse jour/nuit.
- Vitesse : plus lent que SuperPoint sur modèle similaire mais optimisable ; typiquement réalisable à dizaines FPS sur GPU selon backbone et résolution.

5.6 DISK

Principe DISK aborde l'extraction de keypoints comme un problème discret de prise de décision et utilise des techniques d'apprentissage par renforcement (ou des formulations

différentiables) pour apprendre une politique qui sélectionne des positions discrètes de keypoints maximisant l'utilité pour le matching/reconstruction.

Propriétés

- Performance SOTA sur certains benchmarks (grande quantité de matches précis).
- Coût d'entraînement et d'inférence élevé (sélection discrète, mécanismes type MC sampling) ; extraction dense et post-traitements lourds.
- Usage typique : scénarios offline / recherche, moins adapté au temps réel embarqué.

5.7 ALIKE

Principe ALIKE vise à fournir un détecteur+descripteur *accurate & lightweight*. Il combine une architecture multi-échelle efficace et une méthode de localisation subpixelle via *soft-argmax* (similaire à LIFT), en produisant des points subpixelisés et des descripteurs compacts (ex. 128-D).

Performances et usage

- Très rapide : mesures reportées ~ 95 FPS en 640×480 sur GPU courant pour l'ensemble détecteur+descripteur ; architecture pensée pour être légère et efficace.
- Robuste pour la plupart des conditions courantes ; légèrement moins performant que R2D2/D2-Net sur cas extrêmes (jour/nuit très sévères), mais bien meilleur que ORB/BRIEF en général.
- Idéal comme alternative moderne à SuperPoint quand on cherche une meilleure vitesse et une empreinte mémoire plus faible.

5.8 Comparaison synthétique et recommandations pratiques

Résumé (performance vs coût)

- **SuperPoint** : très bon compromis vitesse/robustesse sur GPU (70 FPS 480×640), descripteur 256-D ; excellent point de départ si une accélération matérielle est disponible.
- **ALIKE** : ultra-rapide (95 FPS) et léger — meilleur choix si on privilégie la latence absolue sur GPU embarqué.
- **R2D2** : meilleure robustesse sur localisation extérieure (jour/nuit), descripteur 128-D ; nécessite plus de calcul mais reste utilisable sur GPU.
- **D2-Net** / **DISK** : robustesse maximale pour cas difficiles mais coût d'inférence élevé $\rightarrow$ généralement inadéquat pour embarqué temps réel.
- **TILDE** / **LIFT** : TILDE cible la robustesse aux variations lumineuses (léger), LIFT est conceptuellement complet mais coûteux.

Conseils de déploiement pour l'application ferroviaire

- **Si GPU/NPU embarqué disponible** (NVIDIA Jetson, Coral Edge TPU, NPU), implémenter **SuperPoint** (ou **ALIKE** pour la vitesse maximale). Appliquer quantification (FP16/INT8) et réduire la résolution d'entrée si nécessaire.
- **Si CPU uniquement** : rester sur descripteurs binaires (ORB / BRISK) ou extraire cartes semi-denses de SuperPoint offline pour créer des keyframes/atlas.
- **Matching** : coupler les descripteurs appris avec des méthodes de matching robustes (kNN + ratio-test, cross-check). Pour gains supplémentaires sur taux d'inliers, ajouter **SuperGlue** ou **LightGlue** (network-based matcher) si la ressource matérielle le permet.
- **RANSAC / filtrage géométrique** : indispensable pour estimer homographie / transformation et rejeter outliers. Pour une pipeline temps réel, préférer RANSAC itératif avec seuils adaptatifs et initialisation par matches de haute confiance (reliability map pour R2D2).
- **Tuning dataset-specific** : ré-entraîner ou affiner SuperPoint/R2D2 sur images ferroviaires (MagicPoint-like synthetic + homographic adaptation appliquée à vues de rails) augmente fortement la répétabilité entre jours.

Conclusion technique Les méthodes deep offrent aujourd'hui la meilleure robustesse aux variations photométriques et une très bonne répétabilité. Le choix concret dépendra fortement de la contrainte matérielle : **SuperPoint** / **ALIKE** / **R2D2** sont les options privilégiées sur accélérateur matériel ; **ORB/BRISK** restent la base pragmatique sur CPU seul. Pour un système ferroviaire, une stratégie hybride (ORB en fallback CPU, SuperPoint/ALIKE en mode GPU, ré-entraînement sur données locales et appariement par SuperGlue) fournit une solution robuste et pratique.

6 Comparaison globale des méthodes

Méthode	Type	Invariances	Robustesse	Vitesse	Description	Pertinence ferroviaire
Harris	Détecteur	Rotation	Moyenne	+	-	Pas conseillé, sensible au bruit
FAST	Détecteur	Rotation (approx.)	Moyenne	+++	-	Conseillé, temps réel, peu stable
SIFT	D+Desc.	Échelle, rotation, lumière	Excellente	-	128D	Bon mais trop lent
SURF	D+Desc.	Échelle, rotation	Très bonne	+	64D	Bon mais trop lent

(A)KAZE	D+Desc.	Échelle, rotation, lumière	Très bonne	+	64D	Conseillé, stable sur rails
BRIEF	Desc.	Aucune	Moyenne	+++	Binaire	Pas conseillé, instable
BRISK	D+Desc.	Rotation, échelle	Bonne	+	Binaire	Conseillé, léger mais peut mieux faire
ORB	D+Desc.	Rotation, lumière	Bonne	++	256 bits	Très conseillé, idéal CPU
FREAK	Desc.	Rotation, échelle	Bonne	++	Binaire	Conseillé, bon complément
TILDE	Détecteur DL	Éclairage	Excellente	+	-	Conseillé, bon lumière variable
LIFT	D+Desc. DL	Rotation, échelle, vue	Excellente	-	128D	Pas conseillé, trop lent
SuperPoint	D+Desc. DL	Échelle, rotation modérée, lumière	Très bonne	+	256D	Conseillé seulement si on a un bon GPU
D2-Net	D+Desc. DL	Rotation, lumière	Excellente	-	512D	Pas conseillé, trop lourd
R2D2	D+Desc. DL	Lumière, vue	Excellente	+	128D	Bon compromis
DISK	D+Desc. DL	Vue, rotation, lumière	SOTA	-	128D	Pas conseillé, trop lourd
ALIKE	D+Desc. DL	Rotation modérée, échelle	Bonne	+++	128D	Conseillé, Ultra-rapide

7 Analyse pour le projet ferroviaire

Le système ferroviaire doit répondre à trois contraintes majeures :

1. **Temps réel** : traitement à 30–60 FPS sur matériel embarqué.
2. **Robustesse** : invariance à la lumière, à la perspective et aux conditions climatiques.
3. **Stabilité temporelle** : détection de points répétables sur plusieurs jours.

Les méthodes classiques comme SIFT et SURF, bien que robustes, sont trop lentes. Les approches binaires (ORB, BRISK, FREAK) offrent un excellent compromis entre vitesse et précision. ORB se distingue par sa légèreté et sa stabilité en rotation, ce qui le rend particulièrement adapté aux systèmes embarqués CPU.

Cependant, les conditions lumineuses changeantes du rail (jour/nuit, météo) nécessitent une méthode plus robuste. SuperPoint, grâce à son apprentissage auto-supervisé et à sa robustesse aux variations d'éclairage, représente une solution idéale si un GPU embarqué est disponible. En cas de contraintes de calcul, ALIKE constitue une alternative plus légère tout en restant performante.

Un pipeline optimal pour le recalage ferroviaire pourrait combiner :

- Détection et description via SuperPoint (ou ORB sur CPU).
- Appariement robuste avec SuperGlue ou LightGlue.
- Estimation d'homographie via RANSAC pour l'alignement.

8 Conclusion

Ce rapport a présenté une synthèse complète des principales méthodes de détection et de description de points d'intérêt, depuis les approches classiques jusqu'aux modèles neuronaux récents. Les résultats montrent que :

- Les méthodes traditionnelles comme ORB restent excellentes pour les systèmes embarqués à ressources limitées.
- Les approches profondes comme SuperPoint offrent une robustesse supérieure, notamment aux changements d'éclairage et de perspective, tout en restant compatibles avec une exécution en temps réel sur GPU.
- ALIKE constitue une alternative moderne et légère particulièrement adaptée à la détection rapide en environnement embarqué.

Pour l'application ferroviaire étudiée, **SuperPoint (ou ALIKE) couplé à un appariement robuste (SuperGlue)** représente la solution la plus équilibrée entre vitesse, robustesse et précision pour assurer l'alignement quotidien des images de rails.

Références

- [1] D. Lowe, “Distinctive image features from scale-invariant keypoints,” *International Journal of Computer Vision*, 2004.
- [2] H. Bay, A. Ess, T. Tuytelaars, and L. Van Gool, “Speeded-up robust features (SURF),” *Computer Vision and Image Understanding*, 2008.
- [3] E. Rublee, V. Rabaud, K. Konolige, and G. Bradski, “ORB : An efficient alternative to SIFT or SURF,” in *Proc. ICCV*, 2011.
- [4] D. DeTone, T. Malisiewicz, and A. Rabinovich, “SuperPoint : Self-supervised interest point detection and description,” in *Proc. CVPR*, 2018.
- [5] J. Revaud, P. Weinzaepfel, C. de Souza, N. Pion, G. Csurka, Y. Cabon, and M. Humenberger, “R2D2 : Repeatable and reliable detector and descriptor,” in *Proc. NeurIPS*, 2019.
- [6] M. Dusmanu et al., “D2-Net : A trainable CNN for joint detection and description of local features,” in *Proc. CVPR*, 2019.
- [7] H. Zhao et al., “ALIKE : Accurate and Lightweight Keypoint Detection and Descriptor Extraction,” *IEEE Transactions on Multimedia*, 2022.
- [8] T. Verdie et al., “TILDE : A temporally invariant learned detector,” in *Proc. CVPR*, 2015.
- [9] K. Yi et al., “LIFT : Learned Invariant Feature Transform,” in *Proc. ECCV*, 2016.
- [10] L. Liu et al., “A Review of Keypoints Detection and Feature Description in Image Registration,” *Scientific Programming*, 2021.